DEPARTMENT OF MECHATRONICS

MECHATRONIC SYSTEM INTEGRATION

(MCTA 3203)

SEMESTER 1, 2026/2027

EXPERIMENT 3

SERIAL COMMUNICATION

SECTION 1

GROUP 7

No.	Matric No.	Name
1	2412749	MUHAMMAD ZUHAIL BIN MOHAMED ZUKILAN
2	2419147	MUHAMMAD SYAZWAN BIN SHOIB
3	2412308	NUR EZZATUL SYUHADA BINTI NOR AZIZI
4	2411522	NUR SHAZMIERA BINTI ZUBIR

Date of Submission : 25 October 2026

TABLE OF CONTENT

ABSTRACT 3
PART 1: MOTION DETECTION WITH MPU6050 3
- **1.0 INTRODUCTION AND THEORY** 3
- **1.1 MATERIALS AND EQUIPMENTS** 4
- **1.2 EXPERIMENTAL SETUP AND HARDWARE ASSEMBLY** 4
- **1.3 METHODOLOGY AND PROGRAM LOGIC** 5
- **1.4 DATA COLLECTION AND TECHNICAL ANALYSIS** 9
- **1.5 RESULTS AND ENGINEERING DISCUSSION** 10
- **1.6 CONCLUSION** 11

PART 2: RFID + SERVO ACCESS SYSTEM 11
- **2.0 INTRODUCTION AND THEORY** 11
- **2.1 MATERIALS AND EQUIPMENT** 12
- **2.2 EXPERIMENTAL SETUP AND HARDWARE ASSEMBLY** 12
- **2.3 METHODOLOGY AND PROGRAM LOGIC** 14
- **2.4 DATA COLLECTION AND TECHNICAL ANALYSIS** 19
- **2.5 RESULTS AND ENGINEERING DISCUSSION** 20
- **2.6 CONCLUSION** 20

RECOMMENDATIONS 20
REFERRENCES 21
ACKNOWLEDGEMENT 21
STUDENT DECLARATION 21

ABSTRACT

This experiment presents the development of a motion-activated RFID access control system using an Arduino, an MFRC522 RFID reader, an MPU6050 sensor, a servo motor and red and green LEDs. The experiment was divided into two parts. Part 1 involved collecting accelerometer data from the MPU6050 and visualizing hand movements in real time using Python to analyze circular motion detection.

For part 2, by integrating RFID authentication with motion verification to control access to the system. The RFID reader scanned the card's UID. For authorized cards, the user was required to perform a circular hand motion, which was analysed using sensor data. When both authentication and motion verification were successful, the servo motor moved to the unlocking position, and the green LED turned on. Otherwise, the servo remained in the locking position, and the red LED indicated denied access. This experiment demonstrated the integration of sensor interfacing, real-time data processing, serial communication, and actuator control in an embedded access control system.

PART 1: MOTION DETECTION WITH MPU6050

1.0 INTRODUCTION AND THEORY

The MPU6050 is an inertial measurement sensor that combines a three-axis accelerometer and a three-axis gyroscope to measure acceleration and angular velocity along the X, Y, and Z axes. These measurements allow the movement and orientation of the sensor to be monitored. In this experiment, the MPU6050 is interfaced with an Arduino board through the I²C communication protocol, using the SDA and SCL pins to transfer sensor data.

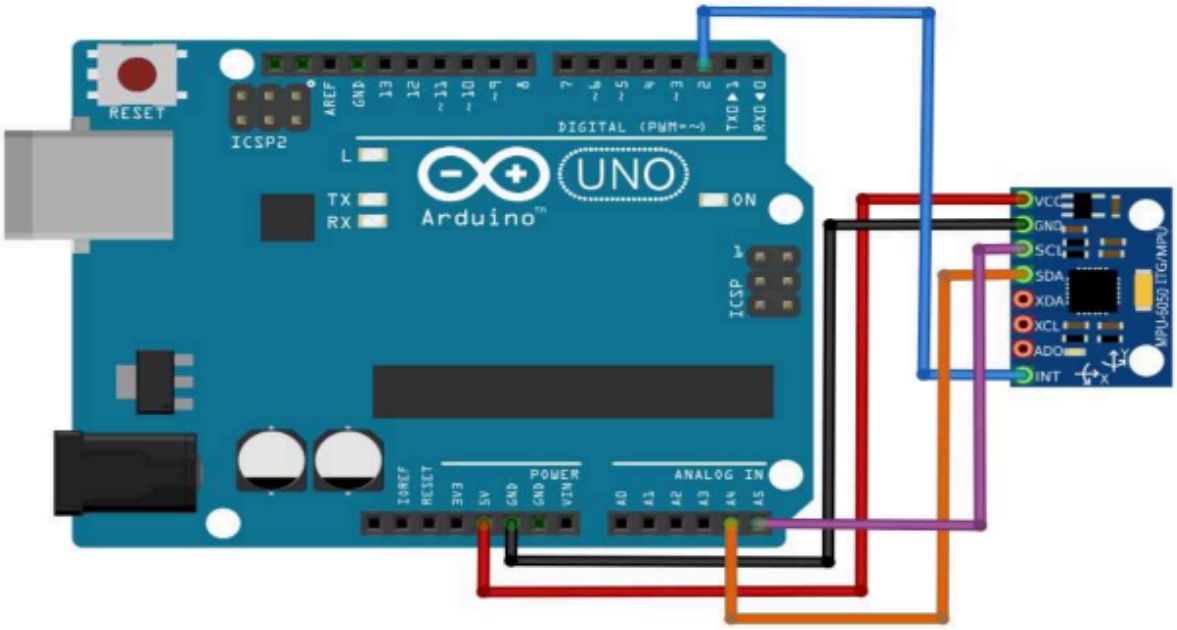
The objective of this experiment is to track hand movements in real time using the MPU6050 sensor and dynamically visualize the motion path in Python. The experiment also aims to detect circular hand gestures by analyzing the accelerometer readings along the X and Y axes. The expected outcomes are successful acquisition of motion data, real-time graphical visualization of the hand movement and identification of circular motion patterns.

The Arduino is programmed using the Arduino IDE to initialize the MPU6050, read accelerometer data and transmit the readings to a computer through USB serial communication at a baud rate of 9600 bits per second. On the computer side, a Python script uses the PySerial library to receive and process the transmitted data while Matplotlib is used to plot the X-Y motion path dynamically. The Python program continuously updates the graph as new readings are received allowing hand movements to be observed and analyzed in real time.

1.1 MATERIALS AND EQUIPMENTS

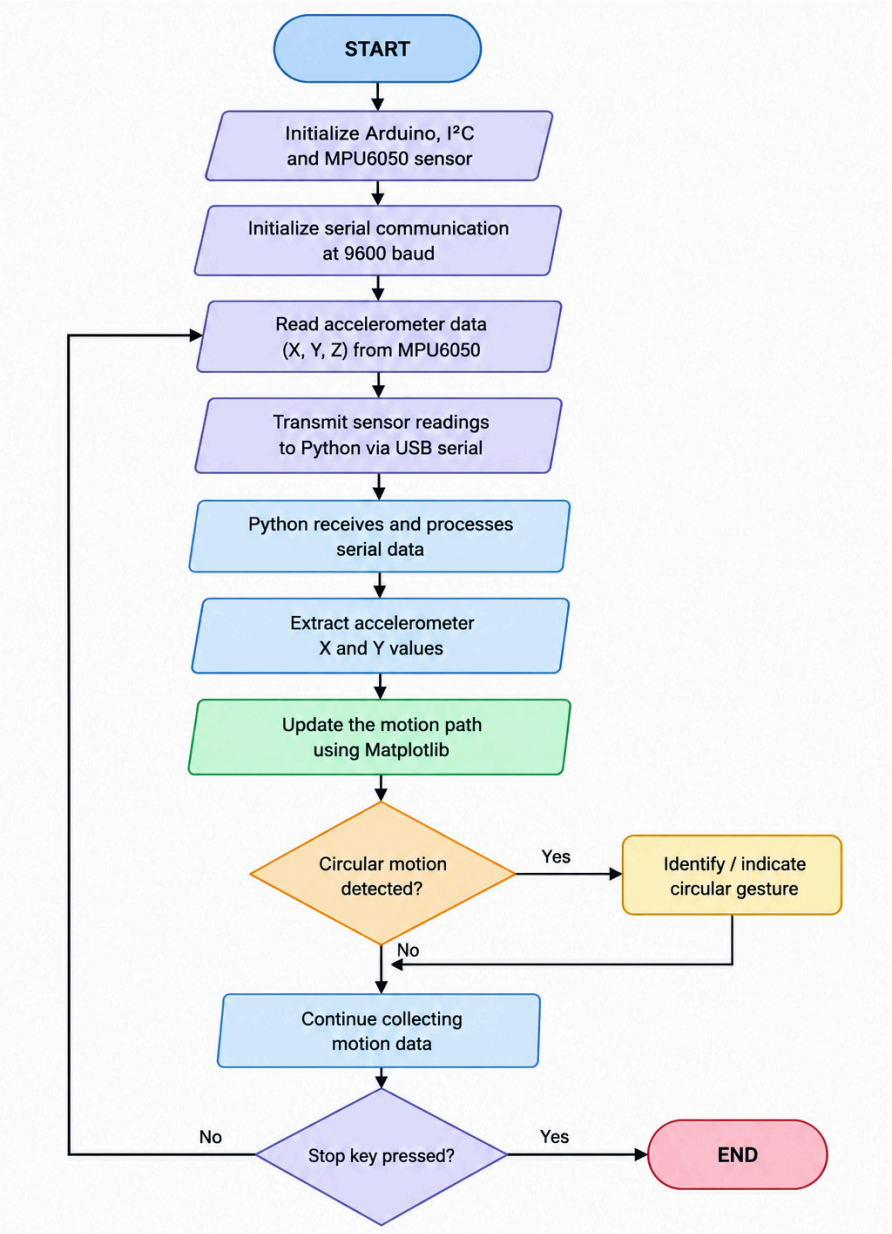
- Arduino board
- MPU6050 sensor
- Jumper wires / breadboard
- USB cable
- Computer with Arduino IDE and Python install
- Arduino IDE
- PyCharm

1.2 EXPERIMENTAL SETUP AND HARDWARE ASSEMBLY



1. VCC pin of MPU6050 was connected to pin 5V arduino.
2. GND pin of MPU6050 was connected to pin GND arduino.
3. SCL pin of MPU6050 was connected to pin A5 arduino.
4. SDA pin of MPU6050 was connected to pin A4 arduino.
5. INT pin of MPU6050 was connected to pin 2 arduino

1.3 METHODOLOGY AND PROGRAM LOGIC



Experiment steps:

1. The Arduino was connected to the computer in the diagram above.
2. The code was uploaded to Arduino and the Python ran.
3. As the X-Y graph appeared, MPU6050 moved in circular motion until the graph detected a circular motion gesture.

IDE code (to read MPU6050 data):

```
#include <Wire.h>
#include <MPU6050.h>
```

```
MPU6050 mpu;
void setup() {
  Serial.begin(9600);
  Wire.begin();
  mpu.initialize();
  if (!mpu.testConnection()) {
    Serial.println("MPU6050 connection failed!");
    while (1);
  }
}
void loop() {
  int ax, ay, az, gx, gy, gz;
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
  Serial.print(ax); Serial.print(",");
  Serial.print(ay); Serial.print(","); Serial.println(az);
  delay(100);
}
```

Python code:

```
from collections import deque
import time
 import serial
import numpy as np

import matplotlib

matplotlib.use('TkAgg')
import matplotlib.pyplot as plt

ser = serial.Serial('COM3', 9600, timeout=1)
time.sleep(2)

WINDOW_SIZE = 100
x_vals = deque(maxlen=WINDOW_SIZE)
y_vals = deque(maxlen=WINDOW_SIZE)

plt.close('all')
```

```
plt.ion()
fig, ax = plt.subplots(figsize=(7, 6))
ax.set_xlim(-20000, 20000)
ax.set_ylim(-20000, 20000)
ax.grid(True)
(line,) = ax.plot([], [], 'ro-', label='Motion Path')

status_text = ax.text(
    0.05,
    0.95,
    '',
    transform=ax.transAxes,
    fontsize=11,
    verticalalignment='top',
    bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.6),
)


def detect_circular_motion(x, y):
    if len(x) < 30:
        return False

    x_arr = np.array(x)
    y_arr = np.array(y)

    cx, cy = np.mean(x_arr), np.mean(y_arr)

    radii = np.sqrt((x_arr - cx) ** 2 + (y_arr - cy) ** 2)
    mean_radius = np.mean(radii)

    if mean_radius > 1000 and (np.std(radii) / mean_radius) < 0.35:
        return True
    return False


print("Starting smooth live motion tracking in external window...")

frame_counter = 0

try:
```

```
  while True:
      if ser.in_waiting > 0:
          raw_data = ser.readline()
          data = raw_data.decode('utf-8', errors='ignore').strip()

          if data:
              values = data.split(',')
              if len(values) >= 3:
                  try:
                      ax_val = int(values[0])
                      ay_val = int(values[1])

                      x_vals.append(ax_val)
                      y_vals.append(ay_val)

                      frame_counter += 1

                      if frame_counter % 3 == 0:
                          line.set_data(list(x_vals), list(y_vals))

                          if detect_circular_motion(x_vals, y_vals):
                              status_text.set_text('Gesture Detected: Circular Motion!')
                              line.set_color('green')
                          else:
                              status_text.set_text('Tracking motion...')
                              line.set_color('red')

                          fig.canvas.draw_idle()
                          fig.canvas.flush_events()

                  except ValueError:
                      continue

  except KeyboardInterrupt:
      print('\nExiting program...')
  finally:
      ser.close()
      plt.ioff()
      plt.show()
```

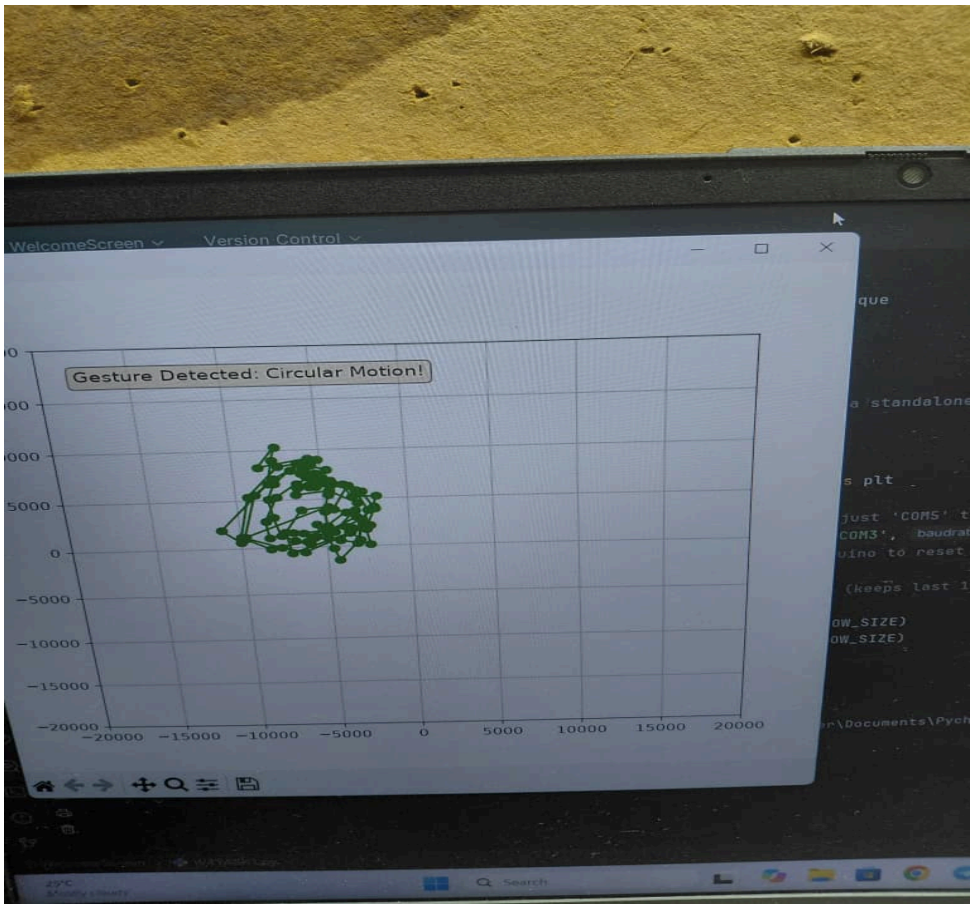
1.4 DATA COLLECTION AND TECHNICAL ANALYSIS

Test	Movement performed with the sensor	Graph behaviour	Detection status
1	Held stationary	Readings cluster around relatively stable region	Not detected
2	Irregular pattern	Inconsistent path	Not detected
3	Repeatedly in circle	Roughly circular or elliptical pattern	Detected if the threshold conditions are satisfied
4	Circle with changing speed or radius	Uneven or distorted	Detection may be inconsistent

Table 1 shows of MPU6050 Motion Tracking and Circular Gesture Detection

The MPU6050 continuously measured acceleration along three axes, and the Arduino transmitted the readings in the format ax, ay, az The Python program separated these values and stored the X-axis and Y-axis readings in two rolling data buffers. The graph was updated every three received readings to reduce the frequency of graphical updates and improve display smoothness. Although the Arduino collected three-axis acceleration data, the Python program only used the X and Y components for plotting and circular motion detection. Therefore, the resulting graph represented the relationship between the two acceleration signals rather than the actual physical position or trajectory of the sensor.

1.5 RESULTS AND ENGINEERING DISCUSSION



The program has several limitations that affect the observation and detection of a complete circular movement :

Limitation	Technical explanation
Limited sampling rate	The Arduino requests readings approximately every 0.1 seconds. A fast circular gesture may be represented by too few points.
Rolling data window	Only the latest 100 readings are retained, so earlier movement gradually disappears from the graph.
Detection does not require a complete circle	The algorithm checks the distribution of distances from the centre, not whether the sensor has completed a full revolution.
Acceleration is not position	The MPU6050 measures acceleration, not displacement. The plotted points therefore do not directly represent the

	path traced by the sensor.
Sensor orientation and gravity	Gravity and changes in sensor orientation affect the measured acceleration, potentially distorting the plotted pattern.

1.6 CONCLUSION

This experiment successfully completed as the gesture detected the circular motion from MPU6050 on the graph. The objective was to establish serial communication between an Arduino Uno and Python which enabled dynamic visualization of 2-axis accelerometer data captured from an MPU-6050 Motion Tracking Module.

PART 2: RFID + SERVO ACCESS SYSTEM

2.0 INTRODUCTION AND THEORY

The RFID and servo access system integrates Radio Frequency Identification (RFID), motion sensing and actuator control to provide an additional layer of access verification. The MFRC522 RFID reader communicates with an RFID card to obtain its unique identification number (UID) which is checked against a list of authorized cards. The MPU6050 sensor is then used to detect a specific hand movement such as a circular gesture. A servo motor acts as the locking mechanism while red and green LEDs provide visual feedback on the access status.

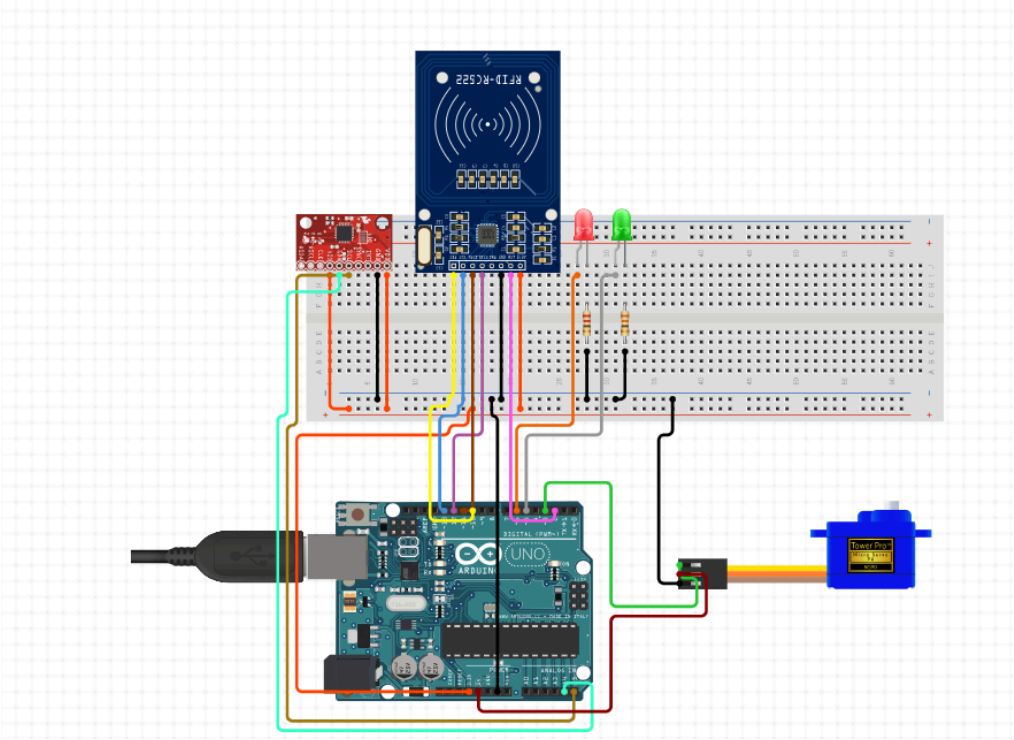
The objective of this experiment is to develop an access control system that combines RFID identification with motion pattern verification. The system is designed to grant access only when an authorized RFID card is detected and the correct motion gesture is performed. The expected outcomes are successful RFID identification, motion verification, servo motor operation and LED indication. If both verification steps are successful, the servo unlocks the system and the green LED turns on. Otherwise, access is denied and the red LED turns on.

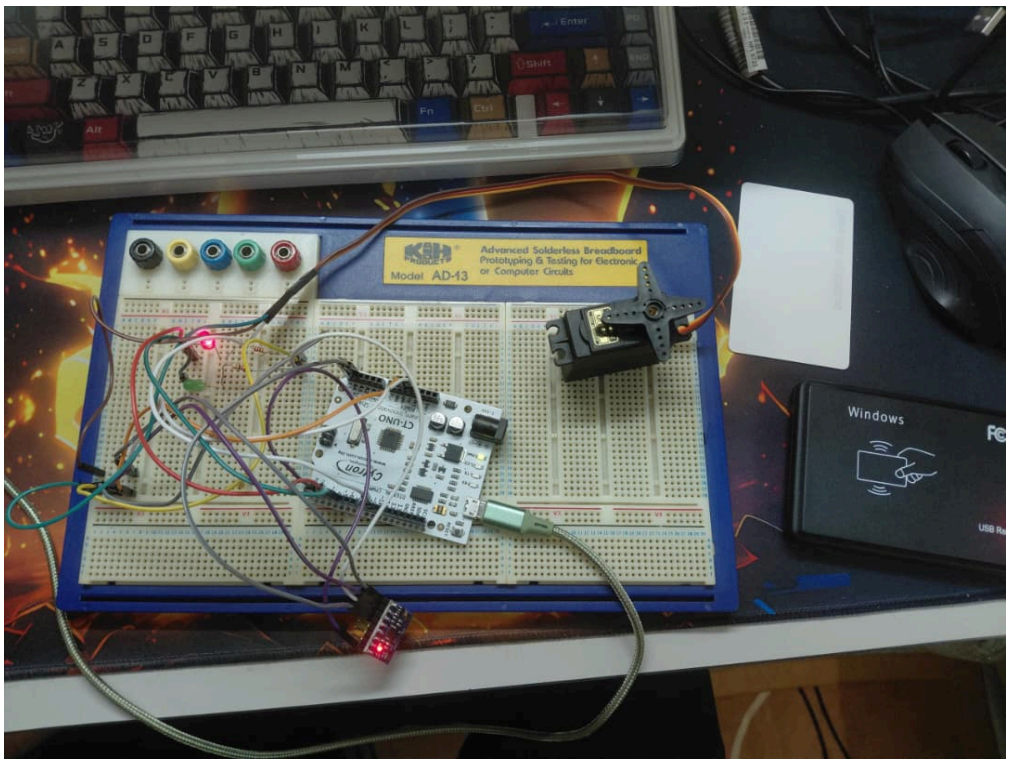
The Arduino is programmed using the Arduino IDE to interface with the MFRC522 RFID reader, servo motor and LEDs. The RFID reader communicates with the Arduino through the Serial Peripheral Interface (SPI) protocol while the MPU6050 uses the Inter-Integrated Circuit (I^2C) protocol to provide motion data. Python is used on the computer to process the RFID identification results and evaluate the detected motion pattern. The computer and Arduino communicate through USB serial communication at a baud rate of 9600 bits per second. Based on the verification result, Python sends an authorization command to the Arduino which controls the servo motor and LEDs accordingly.

2.1 MATERIALS AND EQUIPMENT

- Arduino board
- RFID reader (MFRC522)
- RFID cards
- Servo motor
- Red and Green LEDs
- 220Ω resistors
- MPU6050 sensor
- Jumper wires and breadboard
- Arduino IDE
- PyCharm

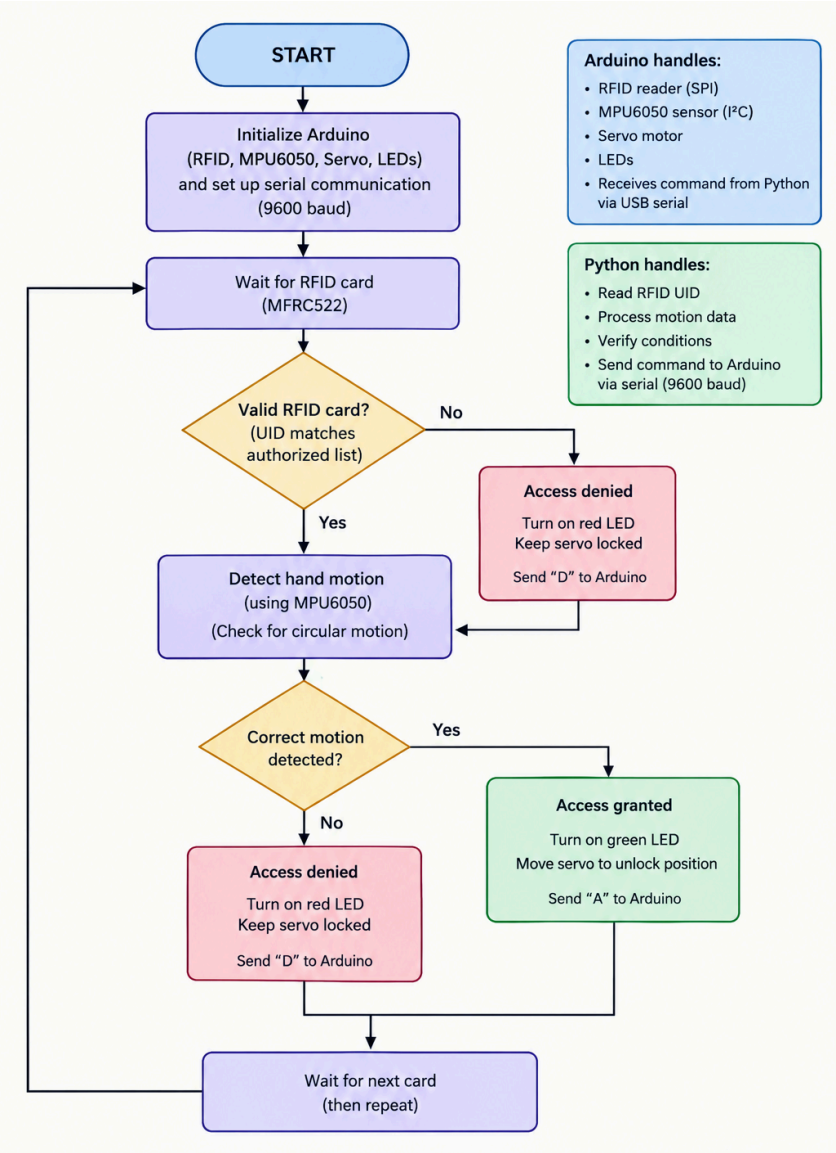
2.2 EXPERIMENTAL SETUP AND HARDWARE ASSEMBLY





1. RFID SS was connected to pin D10 on the Arduino board.
2. RFID RST was connected to pin D8 on the Arduino board.
3. The servo signal was connected to pin D9 on the Arduino board.
4. Green LED was connected to pin D4 on the Arduino board.
5. Red LED was connected to pin D3 on the Arduino board.

2.3 METHODOLOGY AND PROGRAM LOGIC



Experiment steps :

1. The USB RFID scanner is being connected to the computer's USB port.
2. An RFID card was scanned to receive the RFID UID from the Arduino and compare it with the list of authorized UIDs.
3. If the UID was authorized, the user was prompted to perform a circular hand motion using the MPU6050 sensor. Python will analyze the motion data to determine whether the required motion pattern was detected.
4. If both the RFID authentication and circular motion verification were successful, Python sent the character 'A' to the Arduino. The servo motor moved to the unlocking position of 180°, the green LED turned on, and the red LED turned off.

5. If the RFID UID was unauthorized or the required motion was not detected, Python sent the character ‘D’ to the Arduino. The servo motor remained in or returned to the locking position of 90°, the red LED turned on, and the green LED turned off.

IDE code:

```
#include <Wire.h>
#include <Servo.h>
#include <MPU6050.h>

Servo servo;
MPU6050 mpu;

const int servoPin = 9;
const int greenLED = 4;
const int redLED = 3;

void setup() {
  Serial.begin(9600);
  Wire.begin();

  mpu.initialize();

  servo.attach(servoPin);
  servo.write(90);

  pinMode(greenLED, OUTPUT);
  pinMode(redLED, OUTPUT);

  digitalWrite(greenLED, LOW);
  digitalWrite(redLED, HIGH);
}

void loop() {
  int16_t ax, ay, az, gx, gy, gz;
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
```

```
  Serial.print(ax); Serial.print(",");
  Serial.print(ay); Serial.print(",");
  Serial.print(az); Serial.print(",");
  Serial.print(gx); Serial.print(",");
  Serial.print(gy); Serial.print(",");
  Serial.println(gz);

  if (Serial.available() > 0) {
    char command = Serial.read();
    if (command == 'A') {
      servo.write(180);
      digitalWrite(greenLED, HIGH);
      digitalWrite(redLED, LOW);
    } else if (command == 'D') {
      servo.write(90);
      digitalWrite(greenLED, LOW);
      digitalWrite(redLED, HIGH);
    }
  }

  delay(50);
}
```

Python code:

```
import time
import matplotlib.pyplot as plt
import serial

SERIAL_PORT = "COM3"  # Change to match your system
BAUD_RATE = 9600
AUTHORIZED_UID = "0003855091"


def analyze_circular_motion(ser, duration=4):
 """Collects MPU6050 data for a set duration, plots it live,
```

```
and verifies if a circular motion pattern was performed.
"""
print(
    f"Perform circular motion now! Recording for {duration}"
    " seconds..."
)

x_vals, y_vals = [], []
plt.ion()
fig, ax = plt.subplots()
ax.set_xlim(-20000, 20000)
ax.set_ylim(-20000, 20000)
ax.set_title("Live MPU6050 Motion Tracking")
(line,) = ax.plot([], [], "ro-", alpha=0.6)

start_time = time.time()
motion_detected = False

try:
  while time.time() - start_time < duration:
    line_data = ser.readline().decode("utf-8", errors="ignore").strip()
    if line_data:
      values = line_data.split(",")
      if len(values) >= 6:
        try:
          ax_val = int(values[0])
          ay_val = int(values[1])

          x_vals.append(ax_val)
          y_vals.append(ay_val)

          line.set_data(x_vals, y_vals)
          plt.draw()
          plt.pause(0.01)
        except ValueError:
          continue
except KeyboardInterrupt:
  pass

plt.close(fig)
```

```
plt.ioff()

if len(x_vals) > 15:
  x_range = max(x_vals) - min(x_vals)
  y_range = max(y_vals) - min(y_vals)
  if x_range > 3000 and y_range > 3000:
    motion_detected = True

return motion_detected


def main():
try:
  ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
  time.sleep(2)
except Exception as e:
  print(f"Error opening serial port {SERIAL_PORT}: {e}")
  return

print("--- RFID Access Control System Ready ---")

try:
  while True:
    uid = input("Scan RFID tag (or type UID): ").strip()

    if not uid:
      continue

    if uid == AUTHORIZED_UID:
      print("RFID Authorized! Starting motion verification...")

      ser.reset_input_buffer()

      if analyze_circular_motion(ser):
        print("Motion Verified! Access Granted.")
        ser.write(b"A")
      else:
        print("Motion Invalid! Access Denied.")
        ser.write(b"D")
    else:
```

```
      print(f"UID {uid} Unauthorized. Access Denied.")
      ser.write(b"D")

    print("-" * 40)

except KeyboardInterrupt:
  print("\nProgram terminated by user.")
finally:
  if ser.is_open:
    ser.close()


if __name__ == "__main__":
 main()
```

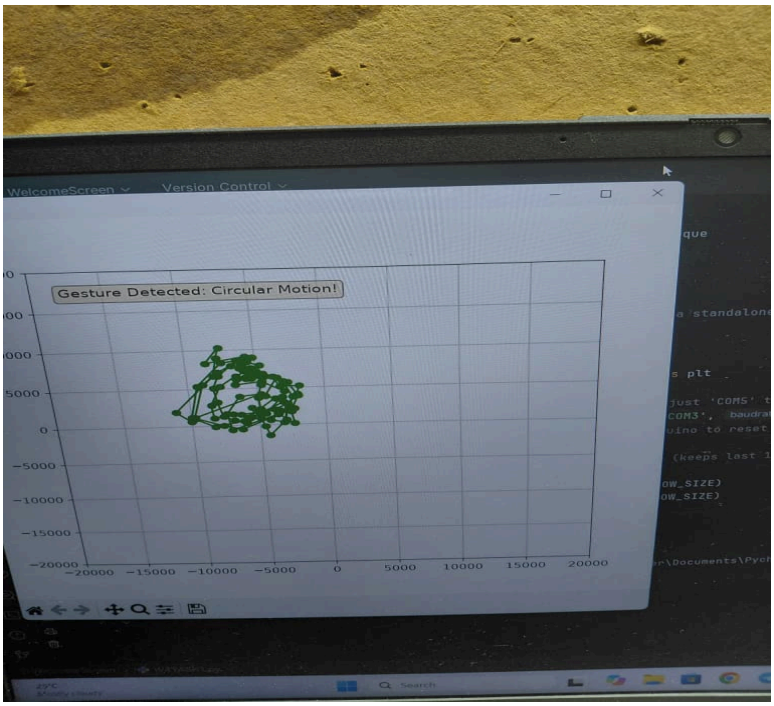
2.4 DATA COLLECTION AND TECHNICAL ANALYSIS

The RFID authentication and motion detection system was tested under different conditions to evaluate its access control functionality. The system response was observed based on the serial commands sent by the Python, servo motor position, and the LED indicators. The results are presented below in Table 2.

Test	RFID status	Motion status	Arduino command	Expected result
1	Authorized	Correct circular motion	A	Green LED ON
2	Authorized	Incorrect motion	D	Red LED ON
3	Unauthorized	Correct circular motion	D	Red LED ON
4	Unauthorized	Incorrect motion	D	Red LED ON

Table 2 shows RFID Authentication and Motion Detection Results.

2.5 RESULTS AND ENGINEERING DISCUSSION



The experiment demonstrated how sensor data, authentication and electronic components could work together in an access control system. However, the motion detection method could incorrectly accept other movements because it only checked the acceleration ranges. Furthermore, the provided code required manual UID input rather than directly reading the RFID card through the MFRC522 module. These limitations should be addressed to improve the system's accuracy and reliability.

2.6 CONCLUSION

The experiment illustrated how multiple electronic components can work together to implement a two-stage access verification process. Nevertheless, the current implementation has limitations because the motion detection algorithm checks acceleration ranges rather than confirming a complete circular gesture, and the provided code does not directly interface with the MFRC522 reader. The servo's programmed angles also differ from the stated intended behaviour. Addressing these limitations would improve the accuracy, security and consistency of the system.

RECOMMENDATIONS

1. Use the appropriate RFID library and SPI communication to read the actual card UID instead of requiring manual keyboard input. The Arduino can then transmit the UID to Python for authentication.

2. Replace the acceleration-range test with an algorithm that evaluates the motion pattern and angular progression. This would reduce the likelihood of accepting unrelated movements as valid circular gestures.

3. Evaluate whether the four-second recording period is sufficient for users to complete the gesture.

4. Add a timeout and reset mechanism. If the user fails to perform the gesture within the allowed time, the system should deny access and return to the initial locked state.

REFERRENCES

ACKNOWLEDGEMENT

We would like to express our deepest appreciation to our instructor and lab personnel for their guidance and support throughout this experiment.

STUDENT DECLARATION

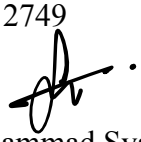
We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been undertaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

Signature: 
Name: Muhammad Zuhail Bin Mohamed Zukilan
Matric : 2412749

Read[/]
Understand[/]
Agree[/]

Signature: 
Name: Muhammad Syazwan Bin Shoib
Matric : 2419147

Read[/]
Understand[/]
Agree[/]

Signature: 
Name: Nur Ezzatul Syuhada Binti Nor Azizi
Matric : 2412308

Read[/]
Understand[/]
Agree[/]

Signature: 
Name: Nur Shazmiera Binti Zubir
Matric : 2411522

Read[/]
Understand[/]
Agree[/]